IC Design Lab
RISC-V Architecture, Group - 4
<u>Modules and Sub Modules with Explanation</u>

Fetch

```
// fetch.v
// fetch_pipe.v
// program_counter.v
// instruc_mem_top.v
```

- **Fetch.v**
  - Top-level fetch stage integration module
  - Instantiates the program counter (PC) module with full hazard control signal routing
  - Memory Control Logic: Combinational logic generates instruction memory interface signals:
    - mask = 4'b1111 (byte mask for 32-bit word access)
    - we_re = 1'b0 (read-only operation for instruction fetch)
    - request = 1'b0 when (load && !valid) to halt instruction fetches during data memory stalls
    - request = 1'b1 otherwise to continuously fetch instructions
  - Data Hazard Handling: Monitors load (load instruction in pipeline) and valid (data memory response) to implement load-use stall mechanism
  - Acts as combinational wrapper that passes through instruction_fetch directly to instruction output

```
// fetch.v
module fetch (
    input wire clk,
    input wire rst,
    input wire next_sel,
    input wire valid,
    input wire load,
    input wire jalr,
    input wire branch_reselt,
    input wire [31:0] next_address,
    input wire [31:0] address_in,
    input wire [31:0] instruction_fetch,
```

```
    output reg we_re,
    output reg request,
    output reg [3:0] mask,
    output wire [31:0] address_out,
    output reg  [31:0] instruction,
    output wire [31:0] pre_address_pc
    );

    // PROGRAM COUNTER
    pc u_pc0
    (
        .clk(clk),
        .rst(rst),
        .load(load),
        .next_sel(next_sel),
        .jalr(jalr),
        .dmem_valid(valid),
        .next_address(next_address),
        .branch_reselt(branch_reselt),
        .address_in(0),
        .address_out(address_out),
        .pre_address_pc(pre_address_pc)
    );

    always @ (*) begin
        if ((load && !valid)) begin
            mask = 4'b1111;
            we_re = 1'b0;
            request = 1'b0;
        end
        else begin
            mask = 4'b1111;
            we_re = 1'b0;
            request = 1'b1;
        end
    end
    always @ (*) begin
        instruction = instruction_fetch ;
    end
endmodule
```

- **Fetch_pipe.v**
    - IF/ID pipeline register with multi-cycle hazard resolution

- State Machine: Uses two-stage flush control with flush_pipeline and flush_pipeline2 flags
- Control Hazard Handling:
  - Primary flush: Inserts NOPs (zeros) when next_select | branch_result | jalr is asserted, sets flush_pipeline flag
  - Secondary flush: Maintains NOP injection for one additional cycle (The module implements a 3-cycle bubble for control hazards)
  - Tertiary flush: One more cycle via flush_pipeline2 (total 3-cycle bubble)
- Load-Use Stall: When load is high, holds current values (pre_address_out, instruction) to stall decode stage
- Normal Operation: Registers pre_address_pc and instruction_fetch on rising clock edge
- All transitions occur on posedge clk with asynchronous active-low reset (negedge rst)

This module is a **fetch pipeline register** that stores the program counter (PC) and fetched instruction between pipeline stages. It **resets or flushes the pipeline** when a branch, jump (jalr), or next instruction change occurs, clearing values for a few cycles.
It uses two flush flags to **stall the pipeline for extra cycles** after a control-flow change.
When a **load hazard** occurs, it stalls by holding the previous PC and instruction.
Otherwise, it **passes the new PC and fetched instruction normally** on each clock cycle.

```
// fetch_pipe.v
module fetch_pipe(
  input wire clk,
  input wire rst,
  input wire [31:0] pre_address_pc,
  input wire [31:0] instruction_fetch,
  input wire next_select,
  input wire branch_result,
  input wire jalr,
  input wire load,

  output wire [31:0] pre_address_out,
  output wire [31:0] instruction
);

  reg [31:0] pre_address, instruc;
  reg flush_pipeline , flush_pipeline2;

  always @ (posedge clk or negedge rst) begin
    if (!rst) begin
      pre_address     <= 32'b0;
      instruc         <= 32'b0;
      flush_pipeline  <= 0;
    end
```

```
    else begin
      if (next_select | branch_result | jalr) begin
      // If jal, jalr, or branch result is high, flush the pipeline
for one cycle
      pre_address     <= 32'b0;
      instruc         <= 32'b0;
      flush_pipeline  <= 1; // Set flag to flush for one cycle
    end
    else begin
      if (flush_pipeline) begin
        // Stall the pipeline for one additional cycle after flushing
        pre_address     <= 32'b0;
        instruc         <= 32'b0;
        flush_pipeline  <= 0; // Reset flag after one cycle stall
        flush_pipeline2 <= 1; // Set flag to flush for one cycle
      end
      else if (flush_pipeline2) begin
        // Stall the pipeline for one additional cycle after flushing
        pre_address     <= 32'b0;
        instruc         <= 32'b0;
        flush_pipeline2 <= 0; // Reset flag after one cycle stall
      end
      else if (load) begin
        //stall pipeline
        pre_address     <= pre_address_out;
        instruc         <= instruction;
      end
      else begin
        // For other instructions, proceed normally
        pre_address     <= pre_address_pc;
        instruc         <= instruction_fetch;
      end
    end
    end
  end

  assign pre_address_out = pre_address;
  assign instruction     = instruc;
endmodule
```

- **Program_counter.v**
  - PC register with priority-based update logic
  - Update Priority (highest to lowest):

        - Branches/JAL: (next_sel | branch_reselt) → PC = next_address (target address from ALU/immediate)
        - JALR: jalr → PC = next_address (register-based jump target)
        - Data Memory Stall: (load && !dmem_valid) → PC = PC (hold current), restore pre_address from saved value
        - Sequential: PC = PC + 4 (default increment)
    - Previous PC Tracking: Maintains pre_address register for PC+4 value needed by JAL/JALR for return address calculation
    - Clocked on posedge clk, resets to address 0 on negedge rst

This module implements a 32-bit Program Counter (PC) that stores the address of the current instruction in a RISC processor. The PC updates on the rising edge of the clock and can either increment normally, jump to a new address, or remain unchanged depending on control signals. It also keeps track of the previous PC value, which is useful for jump and return operations.

When the reset (rst) is low, the PC is cleared to 0, ensuring the processor starts execution from a known address. During normal operation, the current PC value is first saved as the previous PC. If a branch is taken or a jump is requested, the PC is updated with next_address. If a JALR instruction occurs, the PC is also updated with next_address. If the processor is executing a load instruction and the data memory is not yet valid, the PC is held constant to stall instruction fetching. Otherwise, the PC increments by 4 to fetch the next sequential instruction.

The module outputs both the current PC (address_out) and the previous PC (pre_address_pc), allowing proper control flow and return address handling.

1) Reset Condition (Highest Priority)

Condition	PC Action
rst = 0	PC = 0, previous PC cleared

2) PC Update Conditions (Evaluated only when rst = 1)

Priority	Condition	PC Action
1	next_sel = 1 or branch_reselt = 1	PC = next_address
2	jalr = 1	PC = next_address
3	load = 1 and dmem_valid = 0	PC holds value (stall)
4	Otherwise	PC = PC + 4

```
// program_counter
module pc (
    input wire clk,
    input wire rst,
    input wire load,
    input wire jalr,
    input wire next_sel,
    input wire dmem_valid,
    input wire branch_reselt,
    input wire [31:0]next_address,
    input wire [31:0]address_in,

    output reg [31:0]address_out,
    output wire [31:0] pre_address_pc
);

    reg [31:0] pre_address;
    always @(posedge clk or negedge rst) begin
        if(!rst)begin
            address_out <= 0;
            pre_address  <= address_out;
        end

        else begin
            pre_address  <= address_out;
            if (next_sel | branch_reselt)begin
                address_out <= next_address;
            end
            else if (jalr)begin
                address_out <= next_address;
            end
            else if ((load && !dmem_valid))begin
                address_out <= address_out;
                pre_address <= pre_address_pc;
            end
            else begin
                address_out <= address_out + 32'd4;
            end
        end

    end

    assign pre_address_pc = pre_address;
endmodule
```

Instruc_mem_top.v

- Instruction memory controller with 1-cycle latency
- Valid Signal Generation: Registers request signal through one clock cycle to produce valid output (models memory access delay)
- Memory Instantiation: Parameterized memory module with INIT_MEM parameter for memory initialization file
- Interface Signals:
  - 8-bit address bus (256-word address space = 1KB instruction memory)
  - 4-bit mask for byte-level access control (always 4'b1111 for word-aligned instruction fetch)
  - we_re distinguishes read/write operations (always read for instruction memory)
- Synchronous valid generation ensures proper handshaking protocol with fetch stage

This module serves as the top-level wrapper for the processor's instruction storage. It instantiates the generic memory module (u_memory) to store and retrieve program code.

- Wrapper Function: It passes control signals like we_re (write/read enable), mask, and address to the underlying memory block.
- Valid Signal: It includes a sequential logic block that drives the valid output, indicating when the requested instruction is ready (one clock cycle after the request signal is asserted).

Connection to Other Modules:

- Input: Receives the 8-bit address from the program_counter.v (which holds the current PC value).
- Output: Sends the fetched data_out (the 32-bit Instruction) to the fetch_pipe.v (IF/ID Pipeline Register).
- Internal: Directly connects to and wraps the memory.v module, which contains the actual array of data (ROM/RAM) where the instructions are stored.

```
// instruc_memory_top
module instruc_mem_top #(
    parameter INIT_MEM = 0
)(
    input wire clk,
    input wire rst,
    input wire we_re,
    input wire request,
    input wire [3:0]  mask,
    input wire [7:0]  address,
    input wire [31:0] data_in,

    output reg valid,
    output wire [31:0] data_out
    );

    always @(posedge clk or negedge rst ) begin
```

```
        if(!rst)begin
            valid <= 0;
        end
        else begin
            valid <= request;
        end
    end

    memory #(
      .INIT_MEM(INIT_MEM)
    )u_memory(
        .clk(clk),
        .we_re(we_re),
        .request(request),
        .mask(mask),
        .address(address),
        .data_in(data_in),
        .data_out(data_out)
    );
endmodule
```

Decode

```
// Decode.v
// decode_pipe.v
// immediate_gen.v
```

Decode.v

The decode module interprets the fetched instruction and prepares all control signals and operands for the Execute stage. It receives the instruction, PC value, and writeback data, and reads source registers from the register file. The controlunit decodes opcode, funct3, and funct7 to generate signals such as load, store, jalr, reg_write, mem_to_reg, and alu_control. The immediate generator extracts different immediate formats, and a mux selects the required one. The register file provides op_a and op_b. Two muxes select between register values, PC, or immediate to generate final ALU inputs. The branch module compares operands and produces branch_result. In short, this stage converts instruction bits into control signals and prepared operands for execution.

```
// decode
module decode (
    input wire clk,
    input wire rst,
    input wire valid,
    input wire reg_write_en_in,
    input wire load_control_signal,
    input wire [31:0] instruction,
    input wire [31:0] pc_address,
    input wire [31:0] rd_wb_data,
    input wire [31:0] instruction_rd,

    output wire load,
    output wire store,
    output wire jalr,
    output wire next_sel,
    output wire branch_result,
    output wire reg_write_en_out,
    output wire [4:0]  alu_control,
    output wire [1:0]  mem_to_reg,
    output wire [4:0]  rs1 , rs2,
    output wire [31:0] opb_data,
    output wire [31:0] opa_mux_out,
    output wire [31:0] opb_mux_out
    );

    wire branch;
    wire operand_a;
    wire operand_b;
    wire [2:0]  imm_sel;
    wire [31:0] op_a , op_b;
    wire [31:0] imm_mux_out;
    wire [31:0] i_immo , s_immo , sb_immo , uj_immo , u_immo;

    // CONTROL UNIT
    controlunit u_cu0
    (
        .opcode(instruction[6:0]),
        .fun3(instruction[14:12]),
        .fun7(instruction[31:25]),
        .valid(valid),
        .reg_write(reg_write_en_out),
        .imm_sel(imm_sel),
        .next_sel(next_sel),
```

```
    .operand_b(operand_b),
    .operand_a(operand_a),
    .mem_to_reg(mem_to_reg),
    .Load(load),
    .Store(store),
    .jalr_out(jalr),
    .Branch(branch),
    .load_control(load_control_signal),
    .alu_control(alu_control)
);

// IMMEDIATE GENERATION
immediategen u_imm_gen0 (
    .instr(instruction),
    .i_imme(i_immo),
    .sb_imme(sb_immo),
    .s_imme(s_immo),
    .uj_imme(uj_immo),
    .u_imme(u_immo)
);

//IMMEDIATE SELECTION MUX
mux3_8 u_mux0(
    .a(i_immo),
    .b(s_immo),
    .c(sb_immo),
    .d(uj_immo),
    .e(u_immo),
    .sel(imm_sel),
    .out(imm_mux_out)
);

// REGISTER FILE
registerfile u_regfile0
(
    .clk(clk),
    .rst(rst),
    .en(reg_write_en_in),
    .rs1(instruction[19:15]),
    .rs2(instruction[24:20]),
    .rd(instruction_rd[11:7]),
    .data(rd_wb_data),
    .op_a(op_a),
    .op_b(op_b)
);
```

```
    assign rs1 = instruction[19:15];
    assign rs2 = instruction[24:20];
    assign opb_data = op_b ;

    //SELECTION OF PROGRAM COUNTER OR OPERAND A
    mux u_mux1
    (
        .a(op_a),
        .b(pc_address),
        .sel(operand_a),
        .out(opa_mux_out)
    );

    //SELECTION OF OPERAND B OR IMMEDIATE
    mux u_mux2(
        .a(op_b),
        .b(imm_mux_out),
        .sel(operand_b),
        .out(opb_mux_out)
    );

    //BRANCH
    branch u_branch0(
        .en(branch),
        .op_a(op_a),
        .op_b(op_b),
        .fun3(instruction[14:12]),
        .result(branch_result)
    );
endmodule
```

Decode_pipe.v

This module is a **decode pipeline register** that transfers data and control signals from the decode stage to the execute stage of the processor pipeline. It stores information such as register addresses, ALU control signals, operand values, instruction, program counter, and control signals like load, store, branch, jump, and register write.

When the reset signal is active, all stored values are cleared to zero to initialize the pipeline safely. On every rising clock edge, the module captures all input signals from the decode stage and stores them in internal registers. The stored values are then provided as outputs, ensuring that the execute stage receives stable and synchronized data. Like a typical pipeline register, it does not perform stalling or flushing — it simply latches and forwards decode-stage results to the execute stage.

```
// decode_pipe
module  decode_pipe(
  input wire clk,
  input wire rst,
  input wire load_in,
  input wire store_in,
  input wire jalr_in,
  input wire next_sel_in,
  input wire branch_result_in,
  input wire reg_write_in,
  input wire [4:0] rs1_in,
  input wire [4:0] rs2_in,
  input wire [4:0] alu_control_in,
  input wire [1:0]  mem_to_reg_in,
  input wire [31:0] opa_mux_in,
  input wire [31:0] opb_mux_in,
  input wire [31:0] opb_data_in,
  input wire [31:0] pre_address_in,
  input wire [31:0] instruction_in,

  output wire load,
  output wire store,
  output wire jalr_out,
  output wire next_sel,
  output wire branch_result,
  output wire reg_write_out,
  output wire [4:0] rs1_out,
  output wire [4:0] rs2_out,
  output wire [4:0] alu_control,
  output wire [1:0]  mem_to_reg,
  output wire [31:0] opa_mux_out,
  output wire [31:0] opb_mux_out,
  output wire [31:0] opb_data_out,
  output wire [31:0] pre_address_out,
  output wire [31:0] instruction_out
 );

  reg l,s,nextsel,branch_res,jalr;
  reg reg_write;
  reg [1:0] mem_reg;
  reg [4:0] alu_con;
  reg [4:0] rs1;
  reg [4:0] rs2;
  reg [31:0] opa_mux,opb_mux,opb_data,pre_address,instruction;
```

```
always @ (posedge clk or negedge rst) begin
  if (!rst) begin
    l           <= 0;
    s           <= 0;
    jalr        <= 0;
    nextsel     <= 0;
    branch_res  <= 0;
    mem_reg     <= 0;
    alu_con     <= 0;
    opa_mux     <= 0;
    opb_mux     <= 0;
    opb_data    <= 0;
    pre_address <= 0;
    instruction <= 0;
    reg_write   <= 0;
    rs1         <= 0;
    rs2         <= 0;
  end
  else begin
    l           <= load_in;
    s           <= store_in;
    jalr        <= jalr_in;
    nextsel     <= next_sel_in;
    branch_res  <= branch_result_in;
    mem_reg     <= mem_to_reg_in;
    alu_con     <= alu_control_in;
    opa_mux     <= opa_mux_in;
    opb_mux     <= opb_mux_in;
    opb_data    <= opb_data_in;
    pre_address <= pre_address_in;
    instruction <= instruction_in;
    reg_write   <= reg_write_in;
    rs1         <= rs1_in;
    rs2         <= rs2_in;
  end
end

assign load             = l;
assign store            = s;
assign rs1_out          = rs1;
assign rs2_out          = rs2;
assign jalr_out         = jalr;
assign next_sel         = nextsel;
assign reg_write_out    = reg_write;
```

```
  assign branch_result    = branch_res;
  assign mem_to_reg       = mem_reg;
  assign alu_control      = alu_con;
  assign opa_mux_out      = opa_mux;
  assign opb_mux_out      = opb_mux;
  assign opb_data_out     = opb_data;
  assign instruction_out  = instruction;
  assign pre_address_out  = pre_address;
endmodule
```

Immediate_gen.v

Explanation: This combinational logic module is responsible for extracting and sign-extending immediate values embedded within the 32-bit instruction. Since RISC-V instructions scramble immediate bits across different positions, this module simultaneously decodes and reconstructs them into five distinct 32-bit output formats:

- i_imme (I-Type for ALU/Loads)
- s_imme (S-Type for Stores)
- sb_imme (B-Type for Branches)
- uj_imme (J-Type for Jumps)
- u_imme (U-Type for Upper Immediates)

Connection to Other Modules:

- Input: Receives the 32-bit instr from the fetch_pipe.v (IF/ID Pipeline Register).
- Output: Sends all five calculated immediate values (i_imme, s_imme, etc.) to the decode_pipe.v (ID/EX Pipeline Register).
- Logic: These outputs are eventually selected by multiplexers in the Execute Stage (controlled by the Control Unit) to provide the correct operand to the ALU or Branch Adder.

```
// immediate_gen
module immediategen (
    input wire [31:0]instr,
    output reg [31:0] i_imme,
    output reg [31:0] s_imme,
    output reg [31:0] sb_imme,
    output reg [31:0] uj_imme,
    output reg [31:0] u_imme
);

    always @(*) begin
        i_imme  = {{20{instr[31]}}, instr[31:20]};
        s_imme  = {{20{instr[31]}}, instr[31:25], instr[11:7]};
        sb_imme =
{{19{instr[31]}},instr[31],instr[7],instr[30:25],instr[11:8],1'b0};
```

```
        uj_imme =
{{11{instr[31]}},instr[31],instr[19:12],instr[20],instr[30:21],1'b0};
        u_imme  = {{instr[31:12]},12'b0};
    end
endmodule
```

Execute

```
// register_file.v
// exexute.v
// execute_pipe.v
// alu.v
// adder.v
// branch.v
```

register_file.v:

This module **implements a standard 32x32-bit RISC-V Register File,** which acts as the processor's fast active memory with two read ports and one write port. It defines an internal array of 32-bit registers indexed from 1 to 31, purposely omitting index 0 to align with the RISC-V specification where Register 0 (x0) is hardwired to zero. The logic is split into synchronous writing and asynchronous reading: on the rising edge of the clock (if the write-enable signal en is high), the input data is written to the destination rd, whereas read operations occur immediately using combinational logic that checks if the source address (rs1 or rs2) is zero, returning a hardcoded 0 for x0 and the stored array value for all other registers.

```
// register_file
module registerfile (
    input wire clk,
    input wire rst,
    input wire en,
    input wire [4:0]rs1,
    input wire [4:0]rs2,
    input wire [4:0]rd,
    input wire [31:0]data,

    output wire [31:0]op_a,
    output wire [31:0]op_b
);

    reg [31:0] register[31:1];
```

```
    integer i;

    always @(posedge clk or negedge rst) begin
        if(!rst)begin
            for (i=0; i<32; i = i + 1) begin
                register[i] <= 32'b0;
            end
        end
        else begin
            if (en) begin
                register[rd] <= data;
            end
        end
    end

    assign op_a = (rs1 != 0)? register[rs1] : 0;
    assign op_b = (rs2 != 0)? register[rs2] : 0;
endmodule
```

Execute.v:

This execute module represents the **Execute (EX) stage** of a 5-stage pipelined RISC processor (IF–ID–EX–MEM–WB). It takes two 32-bit operands and an alu_control signal to perform arithmetic or logical operations using the ALU. The ALU result (alu_res_out) is used for R-type computations or as an effective address for load/store instructions. The module also includes an adder that calculates the next selected address (next_sel_address), typically used for PC update or branch target computation. This stage is critical because it performs the core computation and address generation required before memory access and write-back stages.

```
// execute
module execute (
    input wire [31:0] a_i,
    input wire [31:0] b_i,
    input wire [4:0]  alu_control,
    input wire [31:0] pc_address,

    output wire [31:0] alu_res_out,
    output wire [31:0] next_sel_address
    );

    // ALU
    alu u_alu0
    (
```

```
        .a_i(a_i),
        .b_i(b_i),
        .op_i(alu_control),
        .res_o(alu_res_out)
    );
    //adder
    adder u_adder0(
        .a(pc_address),
        .adder_out(next_sel_address)
    );
endmodule
```

Execute_pipe.v

This module is an **execute pipeline register** that transfers data and control signals from the execute stage to the next stage of the processor pipeline. It stores values such as the ALU result, instruction, program counter, memory control signals, and register write control.

When the reset signal is active, all stored values are cleared to zero to initialize the pipeline safely. On every rising clock edge, the module captures the incoming signals and stores them in internal registers.The stored values are then provided as outputs, ensuring that the next pipeline stage receives stable and synchronized data. Unlike some other pipeline registers, this module does not perform any stalling or flushing — it simply latches and forwards the execute stage results to the memory stage.

```
// execute_pipe
module execute_pipe(
  input wire clk,
  input wire rst,
  input wire load_in,
  input wire store_in,
  input wire reg_write_in,
  input wire [31:0] opb_datain,
  input wire [31:0] alu_res,
  input wire [1:0] mem_reg_in,
  input wire [31:0] next_sel_addr,
  input wire [31:0] pre_address_in,
  input wire [31:0] instruction_in,

  output wire reg_write_out,
  output wire load_out,
  output wire store_out,
  output wire [31:0] opb_dataout,
  output wire [31:0] alu_res_out,
  output wire [1:0] mem_reg_out,
  output wire [31:0] next_sel_address,
```

```
  output wire [31:0] pre_address_out,
  output wire [31:0] instruction_out
  );

  reg load , store , reg_write;
  reg [1:0] mem_reg;
  reg [31:0] alu_result , nextsel_addr;
  reg [31:0] pre_address , instruction , opb_data;

  always @ (posedge clk or negedge rst) begin
    if (!rst) begin
      load          <= 0;
      store         <= 0;
      mem_reg       <= 0;
      opb_data      <= 0;
      pre_address   <= 0;
      instruction   <= 0;
      alu_result    <= 0;
      nextsel_addr  <= 0;
      reg_write     <= 0;
    end
    else begin
      load          <= load_in;
      store         <= store_in;
      mem_reg       <= mem_reg_in;
      opb_data      <= opb_datain;
      pre_address   <= pre_address_in;
      instruction   <= instruction_in;
      alu_result    <= alu_res;
      nextsel_addr  <= next_sel_addr;
      reg_write     <= reg_write_in;
    end
  end

  assign reg_write_out    = reg_write;
  assign load_out         = load;
  assign store_out        = store;
  assign mem_reg_out      = mem_reg;
  assign opb_dataout      = opb_data;
  assign alu_res_out      = alu_result;
  assign pre_address_out  = pre_address;
  assign instruction_out  = instruction;
  assign next_sel_address = nextsel_addr;
endmodule
```

alu.v

The ALU (Arithmetic Logic Unit) performs all arithmetic, logical, shift, and comparison operations in the **Execute (EX) stage** of the 5-stage RISC-V pipeline.

It takes two 32-bit operands (a_i, b_i) and a 4-bit control signal (op_i) from the control unit. Based on op_i, it performs operations such as add, subtract, AND, OR, XOR, shifts, signed/unsigned comparisons, and LUI pass-through.

The module is fully combinational (always @(*)), so results are generated immediately without waiting for a clock.
Its output is used for arithmetic results, branch decisions, and memory address calculation.

Operation Table

op_i	Operation	RISC-V Instruction Type
0	ADD	add, addi, lw, sw, auipc
1	SUB	sub, beq, bne
10	SLL	sll, slli
11	SLT (signed)	slt, slti
100	SLTU (unsigned)	sltu, sltiu
101	XOR	xor, xori
110	SRL	srl, srli
111	SRA	sra, srai
1000	OR	or, ori
1001	AND	and, andi
1111	PASS B	lui

```
// alu.v
module alu (a_i,b_i,op_i,res_o);

    input wire [31:0]a_i;
    input wire [31:0]b_i;
    input wire [4:0]op_i;
```

```
output reg [31:0]res_o;

reg [63:0] res_64;

always @(*) begin
    if (op_i==5'b00000) begin
        res_o = a_i + b_i; //add
    end
    else if (op_i==5'b00001) begin
        res_o = a_i - b_i; //sub
    end
    else if (op_i==5'b00010) begin
        res_o = a_i << b_i; //shift left logical
    end
    else if (op_i==5'b00011) begin
        res_o = $signed (a_i) < $signed (b_i); //shift less then
    end
    else if (op_i==5'b00100) begin
        res_o = a_i < b_i; //shift less then unsigned
    end
    else if (op_i==5'b00101) begin
        res_o = a_i ^ b_i; //xor
    end
    else if (op_i==5'b00110) begin
        res_o = a_i >> b_i; //shift right logical
    end
    else if (op_i==5'b00111) begin
        res_o = a_i >>> b_i; //shift right arithematic
    end
    else if (op_i==5'b01000) begin
        res_o = a_i | b_i; //or
    end
    else if (op_i==5'b01001) begin
        res_o = a_i & b_i; //and
    end
    else if (op_i==5'b01111) begin
        res_o = b_i; //for lui
    end
    //----------M Extension------------//
    else if (op_i==5'b10000) begin
        res_64= (a_i * b_i);//for MUL
        res_o = res_64[31:0]; //lower bits
    end
    else if (op_i==5'b10001) begin
        res_64= ($signed (a_i) * $signed (b_i));//for MULH
```

```
            res_o = res_64[63:32]; //upper bits
        end
        else if (op_i==5'b10010) begin
            res_64= ($signed (a_i) * (b_i));//for MULHSU
            res_o = res_64[63:32]; //upper bits
        end
        else if (op_i==5'b10011) begin
            res_64= a_i * b_i;     //for MULU
            res_o = res_64[63:32]; //upper bits
        end
        else if (op_i==5'b10100) begin
            res_o = ($signed (a_i) / $signed (b_i)); //for DIV
        end
        else if (op_i==5'b10101) begin
            res_o = a_i / b_i; //for DIVU
        end
        else if (op_i==5'b10110) begin
            res_o = ($signed (a_i) % $signed (b_i)); //for REM
        end
        else if (op_i==5'b10111) begin
            res_o = a_i % b_i; //for REMU
        end
        else begin
            res_o = 0;
        end
    end
endmodule
```

adder.v

The adder module is a combinational circuit used in the **Instruction Fetch (IF) stage** to compute the next sequential Program Counter value.

Since RISC-V instructions are 32-bit (4 bytes), the next instruction address is obtained by:

$$PC_{next} = PC_{current} + 4$$

The module continuously adds a constant value 4 to the input address a and outputs the result adder_out. Because it uses always @(*) and no clock, it synthesizes as pure combinational hardware and allows the processor to fetch one instruction every clock cycle during normal sequential execution.

```
// adder.v
module adder (a,adder_out);
```

```
    input wire [31:0] a;
    output reg [31:0] adder_out;

    always @ (*) begin
        adder_out = a + 32'd4;
    end
endmodule
```

branch.v

The **branch module** in the RV32I pipelined processor is a combinational logic block that evaluates branch conditions in the **Execute (EX) stage** of the 5-stage pipeline. It compares two 32-bit operands (op_a and op_b) based on the branch type specified by the fun3 control signal, which is derived from the instruction encoding. The enable signal (en) ensures that the comparison logic is activated only for valid branch instructions; otherwise, the output is forced to zero. Depending on the value of fun3, the module implements all RV32I branch operations, including equality checks (BEQ, BNE), signed comparisons (BLT, BGE using $signed() casting), and unsigned comparisons (BLTU, BGEU).

The output result is a single-bit signal indicating whether the branch condition is satisfied. This signal is used by the Program Counter (PC) selection logic to decide whether execution continues sequentially or jumps to a branch target address. Since the module is purely combinational (always @(*)), it produces immediate outputs without requiring a clock, allowing branch decisions to be resolved within the EX-stage timing constraints. Overall, the branch module is essential for implementing conditional control flow in the processor.

```
// branch.v
module branch (op_a,op_b,fun3,en,result);

   input [31:0]op_a,op_b;
   input [2:0]fun3;
   input en;

   output reg result;

   always @(*) begin
      if(en==1)begin
         case (fun3)
            3'b000 : result = (op_a == op_b) ? 1 : 0 ;
            3'b001 : result = (op_a != op_b) ? 1 : 0 ;
            3'b100 : result = ($signed (op_a) < $signed (op_b)) ? 1 :
0 ;
            3'b101 : result = ($signed (op_a) >= $signed (op_b)) ? 1
: 0 ;
```

```
            3'b110 : result = (op_a < op_b) ? 1 : 0 ;
            3'b111 : result = (op_a >= op_b) ? 1 : 0 ;
         endcase
      end
      else begin
         result = 0;
      end
   end

endmodule
```

Control

```
// control_unit.v
// control_decoder.v
// type_decoder.v
```

Control_unit.v

This module is the **main control unit** of the processor that generates control signals based on the instruction type and fields. It takes inputs like opcode, function bits (fun3, fun7), and control flags to determine how the instruction should be executed. It first uses a **type decoder** to identify the instruction category (R-type, I-type, load, store, branch, jump, etc.). Then, a **control decoder** uses this instruction type along with function bits to generate detailed control signals.

These control signals determine actions such as register write, memory access, ALU operation, operand selection, branching, and jump control. Overall, the module **coordinates the processor's datapath behavior for each instruction**.

```
// control_unit
module controlunit (
    input wire [6:0] opcode,
    input wire [2:0] fun3,
    input wire [6:0]fun7,
    input wire valid,
    input wire load_control,
```

```
    output wire reg_write,
    output wire [2:0]imm_sel,
    output wire operand_b,
    output wire operand_a,
    output wire [1:0] mem_to_reg,
    output wire Load,
    output wire jalr_out,
    output wire Store,
    output wire Branch,
    output wire mem_en,
    output wire next_sel,
    output wire [4:0] alu_control
);

    wire r_type;
    wire i_type;
    wire load;
    wire store;
    wire branch;
    wire jal;
    wire jalr;
    wire lui;
    wire auipc;

    type_decoder u_typedec0 (
        .opcode(opcode),
        .valid(valid),
        .r_type(r_type),
        .i_type(i_type),
        .load(load),
        .branch(branch),
        .store(store),
        .jal(jal),
        .jalr(jalr),
        .lui(lui),
        .auipc(auipc),
        .load_signal_controller(load_control)
    );

    control_decoder u_controldec0 (
        .fun3(fun3),
        .fun7(fun7),
        .i_type(i_type),
        .r_type(r_type),
        .load(load),
```

```
        .store(store),
        .branch(branch),
        .jal(jal),
        .jalr(jalr),
        .lui(lui),
        .auipc(auipc),
        .next_sel(next_sel),
        .Branch(Branch),
        .Load(Load),
        .Store(Store),
        .jalr_out(jalr_out),
        .mem_to_reg(mem_to_reg),
        .reg_write(reg_write),
        .mem_en(mem_en),
        .operand_b(operand_b),
        .operand_a(operand_a),
        .imm_sel(imm_sel),
        .alu_control(alu_control),
        .load_control(load_control)
    );

endmodule
```

control_decoder.v

Instruction Type	Purpose	Key Control Signals
R-Type	Register-to-register ALU operations	reg_write=1, operand_b=0, mem_to_reg=00, ALU op from fun3/fun7
I-Type (ALU)	ALU operations with immediate	reg_write=1, operand_b=1, imm_sel=000, mem_to_reg=00
Load	Read data from memory to register	reg_write=1, Load=1, operand_b=1, mem_to_reg=01
Store	Write register data to memory	Store=1, mem_en=1, operand_b=1, imm_sel=001

Branch	Conditional control-flow change	Branch=1, operand_a=1, operand_b=1, imm_sel=010
JAL	Unconditional PC-relative jump	reg_write=1, next_sel=1, mem_to_reg=10, imm_sel=011
JALR	Register-indirect jump	reg_write=1, jalr_out=1, operand_b=1, imm_sel=000
LUI	Load upper immediate	reg_write=1, imm_sel=100, alu_control=1111
AUIPC	Add upper immediate to PC	reg_write=1, operand_a=1, operand_b=1, imm_sel=100

The **control_decoder module** is a combinational control unit that generates the necessary control signals for the RV32I processor based on the decoded instruction type and function fields. It interprets inputs such as fun3, fun7, and instruction classification signals (r_type, i_type, load, store, branch, jal, jalr, lui, auipc) to control the datapath. The module determines whether the register file should be written (reg_write), whether memory operations are required (Load, Store, mem_en), how results are routed back (mem_to_reg), and how ALU inputs are selected (operand_a, operand_b). It also selects the appropriate immediate format using imm_sel and specifies the ALU operation via alu_control.

Functionally, the decoder maps instruction encodings to processor behaviour. For ALU instructions (R-type and I-type), it selects operations using fun3 and fun7. For load and store instructions, it enables memory access and address calculation. For branch and jump instructions, it controls PC-related operations and next-address selection. Special instructions like LUI and AUIPC are handled by selecting U-type immediates and appropriate ALU behaviour. Since the module always uses @(*), it operates as pure combinational logic, ensuring immediate generation of control signals without clock dependency, which is essential for proper pipeline execution.

```
// control_decoder
module control_decoder (
    input wire [2:0] fun3,
    input wire [6:0]fun7,
    input wire i_type,
    input wire r_type,
    input wire load,
    input wire store,
    input wire branch,
    input wire jal,
```

```
    input wire jalr,
    input wire lui,
    input wire auipc,
    input wire load_control,

    output reg Load,
    output reg Store,
    output reg jalr_out,
    output reg [1:0] mem_to_reg,
    output reg reg_write,
    output reg mem_en,
    output reg operand_b,
    output reg operand_a,
    output reg [2:0]imm_sel,
    output reg Branch,
    output reg next_sel,
    output reg [4:0]alu_control
);
wire fun_7 = fun7[5];

always @(*) begin
    //reg write signal for register file
    reg_write = r_type | i_type | load | jal | jalr | lui | auipc |
load_control;
    //operand a select for first input of alu
    operand_a = branch | jal | auipc;
    //operand b signal for second input of alu
    operand_b = i_type | load | store | branch | jal | jalr | lui |
auipc;
    //load
    Load = load;
    //store
    Store = store;
    //branch
    Branch =  branch;
    //selection for next address if any jump instrucion run
    next_sel = jal;
    jalr_out = jalr;
    //mem enable
    mem_en = store;

    if(r_type)begin //rtype
        mem_to_reg = 2'b00;
        if(fun3==3'b000 & fun7==0000000)begin
            alu_control = 5'b00000;
```

```
end
else if(fun3==3'b000 & fun7==7'b0100000)begin
    alu_control = 5'b00001;
end
else if (fun3==3'b001 & fun7==7'b0000000)begin
    alu_control = 5'b00010;
end
else if (fun3==3'b010 & fun7==7'b0000000)begin
    alu_control = 5'b00011;
end
else if (fun3==3'b011 & fun7==7'b0000000)begin
    alu_control = 5'b00100;
end
else if (fun3==3'b100 & fun7==7'b0000000)begin
    alu_control = 5'b00101;
end
else if (fun3==3'b101 & fun7==7'b0000000)begin
    alu_control = 5'b00110;
end
else if (fun3==3'b101 & fun7==7'b0100000)begin
    alu_control = 5'b00111;
end
else if (fun3==3'b110 & fun7==7'b0000000)begin
    alu_control = 5'b01000;
end
else if (fun3==3'b111 & fun7==7'b0000000)begin
    alu_control = 5'b01001;
end
//--------M-Extension------------//
else if (fun3==3'b000 & fun7==7'b0000001)begin
    alu_control = 5'b10000;
end
else if (fun3==3'b001 & fun7==7'b0000001)begin
    alu_control = 5'b10001;
end
else if (fun3==3'b010 & fun7==7'b0000001)begin
    alu_control = 5'b10010;
end
else if (fun3==3'b011 & fun7==7'b0000001)begin
    alu_control = 5'b10011;
end
else if (fun3==3'b100 & fun7==0000001)begin
    alu_control = 5'b10100;
end
else if (fun3==3'b101 & fun7==7'b0000001)begin
```

```
        alu_control = 5'b10101;
    end
    else if (fun3==3'b110 & fun7==7'b0000001)begin
        alu_control = 5'b10110;
    end
    else if (fun3==3'b111 & fun7==7'b0000001)begin
        alu_control = 5'b10111;
    end
end
else if (i_type)begin //itype
    imm_sel = 3'b000; //i_type selection
    mem_to_reg = 2'b00;
    if(fun3==3'b000)begin
        alu_control = 5'b00000;
    end
    else if (fun3==3'b001 & fun_7==0)begin
        alu_control = 5'b00010;
    end
    else if (fun3==3'b010 & fun_7==0)begin
        alu_control = 5'b00011;
    end
    else if (fun3==3'b011 & fun_7==0)begin
        alu_control = 5'b00100;
    end
    else if (fun3==3'b100 & fun_7==0)begin
        alu_control = 5'b00101;
    end
    else if (fun3==3'b101 & fun_7==0)begin
        alu_control = 5'b00110;
    end
    else if (fun3==3'b101 & fun_7==1)begin
        alu_control = 5'b00111;
    end
    else if (fun3==3'b110 & fun_7==0)begin
        alu_control = 5'b01000;
    end
    else if (fun3==3'b111 & fun_7==0)begin
        alu_control = 5'b01001;
    end
end
else if (store) begin //store
    imm_sel = 3'b001; //store selection
    mem_to_reg = 2'b00;
    if (fun3==3'b000)begin //sb
        alu_control = 5'b00000;
```

```
        //signal = 2'b00;
    end
    else if (fun3==3'b001)begin //sh
        alu_control = 5'b00000;
        //signal = 2'b01;
    end
    else if (fun3==3'b010)begin //sw
        alu_control = 5'b00000;
        //signal = 2'b10;
    end
end
else if (load) begin
    imm_sel = 3'b000; //i_type selection
    mem_to_reg = 2'b01;
    if (fun3==3'b000)begin //lb
        alu_control = 5'b00000;
    end
    else if(fun3==3'b001)begin //lh
        alu_control = 5'b00000;
    end
    else if(fun3==3'b010)begin //lw
        alu_control = 5'b00000;
    end
    else if(fun3==3'b100)begin //lbu
        alu_control = 5'b00000;
    end
    else if(fun3==3'b101)begin //lhu
        alu_control = 5'b00000;
    end
    else if(fun3==3'b110)begin //lwu
        alu_control = 5'b00000;
    end
end
else if (branch)begin
    alu_control = 5'b00000;
    mem_to_reg = 2'b00;
    imm_sel = 3'b010; //branch selection
end
else if (jal)begin
    alu_control = 5'b00000;
    mem_to_reg = 2'b10;
    imm_sel = 3'b011; //jal selection
end
if(jalr)begin
    mem_to_reg = 2'b00;
```

```
        alu_control = 5'b00000;
        imm_sel = 3'b000;//i_type selection
    end
    else if(lui)begin
        mem_to_reg = 2'b00;
        imm_sel = 3'b100;//u_type selection
        alu_control = 5'b01111;
    end
    else if(auipc)begin
        mem_to_reg = 2'b00;
        alu_control = 5'b00000;
        imm_sel = 3'b100;//u_type selection
    end
end

endmodule
```

type_decoder.v:

This module functions as a combinational instruction decoder for a RISC-V processor, **translating a 7-bit opcode** into specific control flags **that identify the instruction type** (such as R-type, Store, Branch, JAL, etc.). It utilizes an always @(*) block with a case statement to map binary opcodes to one-bit output signals, ensuring all outputs default to zero at the start of the block to prevent latch inference. Uniquely, the module includes safety logic to suppress the load signal; while a standard load opcode (7'b0000011) would typically assert the load flag, this output is gated by the **valid** and **load_signal_controller** inputs. If either of these control signals is high, likely indicating a pipeline stall, hazard, or flush, the load operation is disabled to prevent invalid memory access or state updates.

```
// type_decoder
module type_decoder
(opcode,r_type,i_type,load,store,branch,jal,jalr,lui,auipc,valid,load
_signal_controller);

    input wire [6:0]opcode;
    input wire valid;
    input wire load_signal_controller;

    output reg r_type;
    output reg i_type;
    output reg load;
    output reg store;
    output reg branch;
    output reg jal;
    output reg jalr;
```

```
output reg lui;
output reg auipc;

always @(*)begin
    r_type = 1'b0;
    i_type = 1'b0;
    store = 1'b0;
    load = 1'b0;
    branch = 1'b0;
    auipc = 1'b0;
    jal = 1'b0;
    jalr = 1'b0;
    lui = 1'b0;
    case(opcode)
        7'b0110011:begin
            r_type = 1'b1;
        end
        7'b0010011:begin
            i_type = 1'b1;
        end
        7'b0100011:begin
            store = 1'b1;
        end
        7'b0000011:begin
            if (valid | load_signal_controller) begin
                load = 1'b0;
            end
            else begin
                load = 1'b1;
            end
        end
        7'b1100011:begin
            branch = 1'b1;
        end
        7'b0010111:begin
            auipc = 1'b1;
        end
        7'b1101111:begin
            jal = 1'b1;
        end
        7'b1100111:begin
            jalr = 1'b1;
        end
        7'b0110111:begin
            lui = 1'b1;
```

```
            end

            default:begin
                r_type = 1'b0;
                i_type = 1'b0;
                store= 1'b0;
                load = 1'b0;
                branch = 1'b0;
                auipc = 1'b0;
                jal = 1'b0;
                jalr = 1'b0;
                lui = 1'b0;
            end
        endcase
    end
endmodule
```

Memory

```
// Memory.v
// memory.v
// memstage_pipe.v
// data_memory_top.v
// wrapper_memory_top.v
```

Memory.v

The memory_stage module controls and manages memory access during load and store operations in the processor pipeline. It receives control signals such as load, store, valid, and data_valid from earlier pipeline stages. It takes the ALU-generated address, source register data (op_b), and instruction fields to determine how memory should be accessed. The module uses the wrappermem submodule to format store data, generate byte masks, and process loaded data based on instruction type and address alignment. The wrappermem also handles byte and half-word accesses using the fun3 field of the instruction. The mask output is used to enable specific bytes during memory writes. The store_data_out signal sends properly formatted data to data memory. The wrap_load_out signal sends processed load data back to later stages. The always block generates request and we_re signals to control when memory is accessed and whether it is a read or write. Overall, this module connects the execution stage with data memory and manages data formatting and control for memory operations.

```
// Memory
module memory_stage (
    input rst,
    input wire load,
    input wire store,
    input wire valid,
    input wire data_valid,
    input wire [31:0] op_b,
    input wire [31:0] alu_out_address,
    input wire [31:0] instruction,
    input wire [31:0] wrap_load_in,

    output reg we_re,
    output reg request,
    output wire [3:0]  mask,
    output wire [31:0] store_data_out,
    output wire [31:0] wrap_load_out
    );

    // WRAPPER MEMORY
    wrappermem u_wrap_mem0 (
        .data_i(op_b),
        .byteadd(alu_out_address [1:0]),
        .fun3(instruction [14:12]),
        .mem_en(store),
        .Load(load),
        .wrap_load_in(wrap_load_in),
        .masking(mask),
        .data_valid(data_valid),
        .data_o(store_data_out),
        .wrap_load_out(wrap_load_out)
    );

    always @ (*) begin
        if (!valid & !load & !store) begin
            request = 0;
            we_re = 0;
        end
        else begin
            request = load | store ;
            we_re = store ;
        end
    end
endmodule
```

memory.v

Explanation: This module acts as the generic Random Access Memory (RAM) block for the processor. It is designed to handle both data storage (Data Memory) and instruction storage (Instruction Memory), depending on where it is instantiated.

- Byte-Masked Writes: It supports byte-level write operations using the 4-bit mask input, allowing for granular control (essential for storing bytes, half-words, or full words).
- Synchronous Operation: Both read and write operations occur on the rising edge of clk.
- Dual Mode: The we_re signal toggles between Write (High) and Read (Low) modes when the request is active.
- Initialization: It includes an INIT_MEM parameter that preloads memory from a hex file (useful for loading instructions or initial data during simulation).

Connection to Other Modules:

- Input:
    - Receives address (typically from ALU Result) and data_in (Store Data from Register File) via the execute_pipe.v (EX/MEM Pipeline Register).
    - Receives control signals (we_re, mask, request) derived from the Control Unit.
- Output: Sends data_out (Loaded Data) to the memstage_pipe.v (MEM/WB Pipeline Register).
- Instantiation: This module is often instantiated within wrapper modules such as instruc_mem_top.v (for instructions) or data_memory_top.v (for data) to provide specific interfaces for the Fetch and Memory stages.

```
// memory
module memory#(
    parameter INIT_MEM = 0
)(
    input wire clk,
    input wire we_re,
    input wire request,
    input wire [7:0]address,
    input wire [31:0]data_in,
    input wire [3:0]mask,

    output reg [31:0]data_out
);

    reg [31:0] mem [0:255];

    initial begin
        if (INIT_MEM)
            $readmemh("tb/instr.mem",mem);
    end
```

```
    always @(posedge clk) begin
        if (request && we_re) begin
            if(mask[0]) begin
                mem[address][7:0] <= data_in[7:0];
            end
            if(mask[1]) begin
                mem[address][15:8] <= data_in[15:8];
            end
            if(mask[2]) begin
                mem[address][23:16] <= data_in[23:16];
            end
            if(mask[3]) begin
                mem[address][31:24] <= data_in[31:24];
            end
        end

        else begin
            if (request && !we_re) begin
                data_out <= mem[address];
            end
        end
    end
endmodule
```

memstage_pipe.v

Explanation: This sequential logic module serves as the pipeline register separating the Memory (MEM) stage from the Write Back (WB) stage. Its primary purpose is to synchronize the flow of data and control signals, ensuring that results generated in the Memory stage (such as loaded data or ALU results) remain stable and are passed to the Write Back stage on the next clock cycle.

- buffering: It captures critical signals such as wrap_load_in (Data from Memory), alu_res (ALU Result), and control flags (reg_write, mem_reg) on the rising edge of clk.
- Reset Logic: On a reset (!rst), all outputs are cleared to zero to flush the pipeline.

Connection to Other Modules:

- Input:
    - Receives wrap_load_in (Read Data) from the memory.v (or wrapper_memory.v).
    - Receives alu_res and propagated control signals from the execute_pipe.v (passed through the MEM stage).
- Output: Sends wrap_load_out, alu_res_out, and mem_reg_out to the Write_back.v stage (specifically to Muxes that decide what data to write back to the register file).
- Control: The reg_write_out signal is eventually sent back to the register_file.v in the Decode stage to enable writing the result.

```
// memstage_pipe

module memory_pipe(
  input wire clk,
  input wire rst,
  input wire reg_write_in,
  input wire [1:0]  mem_reg_in,
  input wire [31:0] wrap_load_in,
  input wire [31:0] alu_res,
  input wire [31:0] next_sel_addr,
  input wire [31:0] instruction_in,
  input wire [31:0] pre_address_in,

  output wire reg_write_out,
  output wire [31:0] alu_res_out,
  output wire [1:0]  mem_reg_out,
  output wire [31:0] next_sel_address,
  output wire [31:0] wrap_load_out,
  output wire [31:0] instruction_out,
  output wire [31:0] pre_address_out
  );

  reg reg_write;
  reg [1:0]  mem_reg;
  reg [31:0] pre_address_pc;
  reg [31:0] alu_result , nextsel_addr , wrap_load , instruction;

  always @ (posedge clk) begin
    if (!rst) begin
      mem_reg        <= 0;
      alu_result     <= 0;
      nextsel_addr   <= 0;
      wrap_load      <= 0;
      instruction    <= 0;
      reg_write      <= 0;
      pre_address_pc <= 0;
    end
    else begin
      mem_reg        <= mem_reg_in;
      alu_result     <= alu_res;
      nextsel_addr   <= next_sel_addr;
      wrap_load      <= wrap_load_in;
      instruction    <= instruction_in;
      reg_write      <= reg_write_in;
      pre_address_pc <= pre_address_in;
```

```
    end
  end

  assign reg_write_out    = reg_write;
  assign mem_reg_out      = mem_reg;
  assign alu_res_out      = alu_result;
  assign next_sel_address = nextsel_addr;
  assign wrap_load_out    = wrap_load;
  assign instruction_out  = instruction;
  assign pre_address_out  = pre_address_pc;
endmodule
```

Data_memory_top.v

This module is the **top-level data memory interface** that connects the processor to the memory block. It manages memory read/write operations and controls the validity of the output data. When reset is active, the **valid signal is cleared** to indicate no data is ready. On each clock cycle, the valid signal becomes equal to the load signal, meaning it indicates when a memory read operation is active and data is ready to be used.

The module also instantiates an internal **memory block**, passing signals such as write/read enable, address, mask, and input data to it. The memory returns the requested data through data_out, while this top module mainly handles control and output validity signaling.

```
// data_memory_top
module data_mem_top #(
    parameter INIT_MEM = 0
)(
    input wire clk,
    input wire rst,
    input wire we_re,
    input wire request,
    input wire load,
    input wire [3:0]  mask,
    input wire [7:0]  address,
    input wire [31:0] data_in,

    output reg valid,
    output wire [31:0] data_out
    );

    always @(posedge clk or negedge rst ) begin
        if(!rst)begin
```

```
            valid <= 0;
        end
        else begin
            valid <= load;
        end
    end

    memory #(
      .INIT_MEM(INIT_MEM)
    )u_memory(
        .clk(clk),
        .we_re(we_re),
        .request(request),
        .mask(mask),
        .address(address),
        .data_in(data_in),
        .data_out(data_out)
    );
endmodule
```

wrapper_memory.v:

This module acts as the **Load/Store Data Alignment Unit** for a RISC-V processor. Since the data memory is typically 32 bits wide, but RISC-V allows reading/writing just 8 bits (Byte) or 16 bits (Half-word), this module handles the necessary data shifting and masking to ensure the correct bytes are accessed.

1. Store Logic (Writing to Memory)

When mem_en is High, the module prepares data to be written to the RAM.

- **Byte Steering:** The input data (data_i) usually sits in the lowest bits of a register. If the address (byteadd) points to a different location (like the 2nd byte of a word), this logic shifts the data to the correct lane (e.g., bits [15:8]).
- **Write Masking (masking):** It generates a 4-bit signal to tell the RAM which specific bytes to overwrite.
    - **SB (Store Byte):** Activates only 1 bit in the mask (e.g., 0010) based on byteadd.
    - **SH (Store Half):** Activates 2 bits (e.g., 1100).
    - **SW (Store Word):** Activates all 4 bits (1111).

2. Load Logic (Reading from Memory)

When Load or data_valid is High, the module processes raw 32-bit data (wrap_load_in) coming from memory.

- **Extraction:** It uses byteadd to pick the specific byte or half-word the user requested from the full 32-bit word.
- **Extension (Signed vs. Unsigned):** It expands the extracted data back to 32 bits to fit into the destination register.
    - **Signed (LB, LH):** It performs **Sign Extension** (replicating the most significant bit to the left) to preserve negative values.
    - **Unsigned (LBU, LHU):** It performs **Zero Extension** (filling upper bits with 0s).

```
// wrapper_memory
module wrappermem (
    input wire [31:0] data_i,
    input wire [1:0] byteadd,
    input wire [2:0] fun3,
    input wire mem_en,
    input wire Load,
    input wire data_valid,
    input wire [31:0]wrap_load_in,

    output reg [3:0] masking,
    output reg [31:0] data_o,
    output reg [31:0] wrap_load_out

);

    always @(*) begin
        if (mem_en) begin
            masking = 4'b0000;
            if(fun3==3'b000)begin //sb
               case (byteadd)
                    00: begin
                        masking = 4'b0001;
                        data_o = data_i;
                    end
                    01: begin
                        masking = 4'b0010;
                        data_o =
{data_i[31:16],data_i[7:0],data_i[7:0]};
                    end
                    10: begin
                        masking = 4'b0100;
                        data_o =
{data_i[31:24],data_i[7:0],data_i[15:0]};
                    end
                    11: begin
                        masking = 4'b1000;
```

```
                        data_o = {data_i[7:0],data_i[23:0]};
                    end
                endcase
            end

            if(fun3==3'b001)begin //sh
                case (byteadd)
                    00: begin
                        masking = 4'b0011;
                        data_o = data_i;
                    end
                    01: begin
                        masking = 4'b0110;
                        data_o =
{data_i[31:24],data_i[15:0],data_i[7:0]};
                    end
                    10: begin
                        masking = 4'b1100;
                        data_o = {data_i[15:0],data_i[15:0]};
                    end
                endcase
            end

            if(fun3==3'b010) begin //sw
                masking = 4'b1111;
                data_o = data_i;
            end
        end

        if (Load | data_valid)begin
            if(fun3==3'b000)begin //lb
                case (byteadd)
                    00: begin
                        wrap_load_out =
{{24{wrap_load_in[7]}},wrap_load_in[7:0]};
                    end
                    01: begin
                        wrap_load_out =
{{24{wrap_load_in[15]}},wrap_load_in[15:8]};
                    end
                    10: begin
                        wrap_load_out =
{{24{wrap_load_in[23]}},wrap_load_in[23:16]};
                    end
                    11: begin
```

```
                        wrap_load_out =
{{24{wrap_load_in[31]}},wrap_load_in[31:24]};
                    end
               endcase
            end

            if(fun3==3'b001)begin //lh
                case (byteadd)
                    00: begin
                        wrap_load_out =
{{16{wrap_load_in[15]}},wrap_load_in[15:0]};
                    end
                    01: begin
                        wrap_load_out =
{{16{wrap_load_in[23]}},wrap_load_in[23:8]};
                    end
                    10: begin
                        wrap_load_out =
{{16{wrap_load_in[31]}},wrap_load_in[31:16]};
                    end
                endcase
            end

            if(fun3==3'b010) begin //lw
                wrap_load_out = wrap_load_in;
            end

            if(fun3==3'b100)begin //lbu
                case (byteadd)
                    00: begin
                        wrap_load_out = {24'b0,wrap_load_in[7:0]};
                    end
                    01: begin
                        wrap_load_out = {24'b0,wrap_load_in[15:8]};
                    end
                    10: begin
                        wrap_load_out = {24'b0,wrap_load_in[23:16]};
                    end
                    11: begin
                        wrap_load_out = {24'b0,wrap_load_in[31:24]};
                    end
               endcase
            end

            if(fun3==3'b101)begin //lhu
```

```
                case (byteadd)
                    00: begin
                        wrap_load_out = {16'b0,wrap_load_in[15:0]};
                    end
                    01: begin
                        wrap_load_out = {16'b0,wrap_load_in[23:8]};
                    end
                    10: begin
                        wrap_load_out = {16'b0,wrap_load_in[31:16]};
                    end
                endcase
            end

            if(fun3==3'b110) begin //lwu
                wrap_load_out = wrap_load_in;
            end
        end
    end
endmodule
```

Write back, common modules and Main processor

```
// write_back.v
// mux1_2.v
// mux2_4.v
// mux3_8.v
```

Write_back.v

The **Write-Back (WB) stage** is the final stage of the 5-stage pipelined RV32I processor, where the processor writes the correct result into the destination register (**rd**). Since different instruction types produce results from different sources, this block's job is to select the appropriate value and forward it to the register file.

For **arithmetic and logical instructions**, the result comes from the **ALU (alu_out)**. For **load instructions**, the value to be written back is the data fetched from memory **(data_mem_out)**. For **jump and link instructions** such as **JAL** and **JALR**, the processor must store the return address (typically PC + 4), provided here as **next_sel_address**.

The module uses a multiplexer controlled by the 2-bit control signal **mem_to_reg** to choose one of these inputs. The selected value is output as **rd_sel_mux_out**, which is then written into the destination register.

Thus, the write-back stage ensures that each instruction updates the register file with the correct final result, allowing subsequent instructions in the pipeline to use accurate data.

```
// write back
module write_back(
    input wire [1:0]  mem_to_reg,
    input wire [31:0] alu_out,
    input wire [31:0] data_mem_out,
    input wire [31:0] next_sel_address,

    output wire [31:0] rd_sel_mux_out
    );

    //WRITE BACK MUX
    mux2_4 u_mux2 (
        .a(alu_out),
        .b(data_mem_out),
        .c(next_sel_address),
        .sel(mem_to_reg),
        .out(rd_sel_mux_out)
    );
endmodule
```

mux1_2.v

This module implements a 32-bit 2:1 multiplexer with two 32-bit data inputs (a and b), one 1-bit select input (sel), and one 32-bit output (out)
Operation:
sel = 0 → out = a
sel = 1 → out = b

The select signal sel determines which of the two inputs is routed to the output.

```
// mux1_2
module mux (a,b,sel,out);
    input wire [31:0] a,b;
    input wire sel;

    output wire [31:0] out;
```

```
    assign out = (sel)?  b:a;
endmodule
```

mux2_4.v

This module implements a 32-bit 4:1 multiplexer with four 32-bit inputs (a, b, c, d), a 2-bit select line (sel), and a 32-bit output (out), where the value of sel selects which input appears at the output.
Operation:

- sel = 2'b00 → out = a
- sel = 2'b01 → out = b
- sel = 2'b10 → out = c
- sel = 2'b11 → out = d

The 2-bit select signal sel determines which of the four inputs is routed to the output.

```
// mux2_4
module mux2_4 (a,b,c,d,sel,out);
    input wire [31:0] a,b,c,d;
    input wire [1:0] sel;

    output reg [31:0] out;

    always @ (*) begin
        case (sel)
            2'b00 : out = a;
            2'b01 : out = b;
            2'b10 : out = c;
            2'b11 : out = d;
        endcase
    end
endmodule
```

mux3_8.v

This module implements a 32-bit 8:1 multiplexer with eight 32-bit inputs (a to h), a 3-bit select line (sel), and a 32-bit output (out), where the value of sel selects the corresponding input to appear at the output.
Operation:

- sel = 3'b000 → out = a
- sel = 3'b001 → out = b
- sel = 3'b010 → out = c
- sel = 3'b011 → out = d

- sel = 3'b100 → out = e
- sel = 3'b101 → out = f
- sel = 3'b110 → out = g
- sel = 3'b111 → out = h

The 3-bit select signal sel determines which of the eight inputs is routed to the output.

```
// mux3_8
module mux3_8 (a,b,c,d,e,f,g,h,sel,out);
    input wire [31:0] a,b,c,d,e,f,g,h;
    input wire [2:0] sel;

    output reg [31:0] out;

    always @ (*) begin
        case (sel)
            3'b000 : out = a;
            3'b001 : out = b;
            3'b010 : out = c;
            3'b011 : out = d;
            3'b100 : out = e;
            3'b101 : out = f;
            3'b110 : out = g;
            3'b111 : out = h;
        endcase
    end
endmodule
```

```
// STAGE 7 : Core and Microprocessor
%%%
```

Core.v

The core module is the central datapath of the 5-stage RV32I pipelined processor. It connects Fetch, Decode, Execute, Memory, and Writeback stages and manages signal flow between them. It receives instructions from instruction memory and load data from data memory, and produces control signals for memory access and register write-back.

Inputs include clock, reset, instruction data, memory valid signals, and load data. Outputs include PC address, ALU result (memory address), store data, memory request signals, write-enable signals, and byte masks.

Signal flow:

- Fetch stage gets instruction using PC.
- Decode stage generates control signals and reads registers.
- Execute stage performs ALU operations and branch calculation.
- Memory stage handles load/store and masking.
- Writeback stage selects final data for register file.

It also includes basic forwarding logic to reduce data hazards.

In short, this module coordinates the entire pipeline and executes instructions by controlling data movement across all stages.

```
// core
module core (
    input wire clk,
    input wire rst,
    input wire data_mem_valid,
    input wire instruc_mem_valid,
    input wire [31:0] instruction,
    input wire [31:0] load_data_in,

    output wire load_signal,
    output wire instruction_mem_we_re,
    output wire instruction_mem_request,
    output wire data_mem_we_re,
    output wire data_mem_request,
    output wire [3:0]  mask_singal,
    output wire [3:0]  instruc_mask_singal,
    output wire [31:0] store_data_out,
    output wire [31:0] alu_out_address,
    output wire [31:0] pc_address
```

```
    );

    wire [31:0] instruc_data_out;
    wire [31:0] pre_address_pc;
    wire [31:0] instruction_fetch , instruction_decode ,
instruction_execute;
    wire [31:0] instruction_memstage , instruction_wb;
    wire [31:0] pre_pc_addr_fetch , pre_pc_addr_decode ,
pre_pc_addr_execute;
    wire [31:0] pre_pc_addr_memstage , pre_pc_addr_wb;
    wire load_decode , load_execute , load_memstage;
    wire store_decode , store_execute , store_memstage;
    wire jalr_decode;
    wire next_sel_decode , next_sel_execute;
    wire reg_write_decode , reg_write_execute , reg_write_memstage;
    wire branch_result_decode , branch_result_execute;
    wire [3:0]  mask;
    wire [4:0]  alu_control_decode , alu_control_execute;
    wire [1:0]  mem_to_reg_decode , mem_to_reg_execute;
    wire [1:0]  mem_to_reg_memstage , mem_to_reg_wb;
    wire [4:0]  rs1_decode , rs1_execute;
    wire [4:0]  rs2_decode , rs2_execute;
    wire [4:0]  rd_memstage , rd_wb;
    wire [31:0] op_b_decode , op_b_execute , op_b_memstage;
    wire [31:0] opa_mux_out_decode , opa_mux_out_execute;
    wire [31:0] opb_mux_out_decode , opb_mux_out_execute;
    wire [31:0] alu_res_out_execute , alu_res_out_memstage;
    wire [31:0] alu_res_out_wb;
    wire [31:0] next_sel_address_execute , next_sel_address_memstage;
    wire [31:0] next_sel_address_wb;
    wire [31:0] wrap_load_memstage , wrap_load_wb;
    wire [31:0] rd_wb_data;
    wire [31:0] alu_in_a , alu_in_b;

    //FETCH STAGE
    fetch u_fetchstage(
        .clk(clk),
        .rst(rst),
        .load(load_decode),
        .jalr(jalr_execute),
        .next_sel(next_sel_execute),
        .branch_reselt(branch_result_execute),
        .next_address(alu_res_out_execute),
        .instruction_fetch(instruction),
        .instruction(instruction_fetch),
```

```
    .address_in(0),
    .valid(data_mem_valid),
    .mask(instruc_mask_singal),
    .we_re(instruction_mem_we_re),
    .request(instruction_mem_request),
    .pre_address_pc(pre_pc_addr_fetch),
    .address_out(pc_address)
);

//FETCH STAGE PIPELINE
fetch_pipe u_fetchpipeline(
    .clk(clk),
    .rst(rst),
    .pre_address_pc(pre_pc_addr_fetch),
    .instruction_fetch(instruction_fetch),
    .pre_address_out(pre_pc_addr_decode),
    .instruction(instruction_decode),
    .next_select(next_sel_decode),
    .branch_result(branch_result_decode),
    .jalr(jalr_decode),
    .load(load_decode)
);

//DECODE STAGE
decode u_decodestage(
    .clk(clk),
    .rst(rst),
    .valid(data_mem_valid),
    .load_control_signal(load_execute),
    .reg_write_en_in(reg_write_wb),
    .instruction(instruction_decode),
    .pc_address(pre_pc_addr_decode),
    .rd_wb_data(rd_wb_data),
    .rs1(rs1_decode),
    .rs2(rs2_decode),
    .load(load_decode),
    .store(store_decode),
    .jalr(jalr_decode),
    .next_sel(next_sel_decode),
    .reg_write_en_out(reg_write_decode),
    .mem_to_reg(mem_to_reg_decode),
    .branch_result(branch_result_decode),
    .opb_data(op_b_decode),
    .instruction_rd(instruction_wb),
    .alu_control(alu_control_decode),
```

```
        .opa_mux_out(opa_mux_out_decode),
        .opb_mux_out(opb_mux_out_decode)
    );

    //DECODE STAGE PIPELINE
    decode_pipe u_decodepipeline(
        .clk(clk),
        .rst(rst),
        .load_in(load_decode),
        .store_in(store_decode),
        .jalr_in(jalr_decode),
        .next_sel_in(next_sel_decode),
        .mem_to_reg_in(mem_to_reg_decode),
        .branch_result_in(branch_result_decode),
        .opb_data_in(op_b_decode),
        .alu_control_in(alu_control_decode),
        .opa_mux_in(opa_mux_out_decode),
        .opb_mux_in(opb_mux_out_decode),
        .pre_address_in(pre_pc_addr_decode),
        .instruction_in(instruction_decode),
        .reg_write_in(reg_write_decode),
        .rs1_in(rs1_decode),
        .rs2_in(rs2_decode),
        .reg_write_out(reg_write_execute),
        .jalr_out(jalr_execute),
        .load(load_execute),
        .store(store_execute),
        .next_sel(next_sel_execute),
        .mem_to_reg(mem_to_reg_execute),
        .branch_result(branch_result_execute),
        .opb_data_out(op_b_execute),
        .alu_control(alu_control_execute),
        .opa_mux_out(opa_mux_out_execute),
        .opb_mux_out(opb_mux_out_execute),
        .pre_address_out(pre_pc_addr_execute),
        .instruction_out(instruction_execute),
        .rs1_out(rs1_execute),
        .rs2_out(rs2_execute)
    );

    assign alu_in_a = (rs1_execute == rd_memstage )?
alu_res_out_memstage :
                      (rs1_execute == rd_wb       )? alu_res_out_wb :
opa_mux_out_execute;
```

```
    assign alu_in_b = (rs2_execute == rd_memstage )?
alu_res_out_memstage :
                      (rs2_execute == rd_wb       )? alu_res_out_wb :
opb_mux_out_execute;

    //EXECUTE STAGE
    execute u_executestage(
        .a_i(alu_in_a),
        .b_i(alu_in_b),
        .pc_address(pre_pc_addr_execute),
        .alu_control(alu_control_execute),
        .alu_res_out(alu_res_out_execute),
        .next_sel_address(next_sel_address_execute)
    );

    //EXECUTE STAGE PIPELINE
    execute_pipe u_executepipeline(
        .clk(clk),
        .rst(rst),
        .load_in(load_execute),
        .store_in(store_execute),
        .opb_datain(op_b_execute),
        .alu_res(alu_res_out_execute),
        .mem_reg_in(mem_to_reg_execute),
        .next_sel_addr(next_sel_address_execute),
        .pre_address_in(pre_pc_addr_execute),
        .instruction_in(instruction_execute),
        .reg_write_in(reg_write_execute),
        .reg_write_out(reg_write_memstage),
        .load_out(load_memstage),
        .store_out(store_memstage),
        .opb_dataout(op_b_memstage),
        .alu_res_out(alu_res_out_memstage),
        .mem_reg_out(mem_to_reg_memstage),
        .next_sel_address(next_sel_address_memstage),
        .pre_address_out(pre_pc_addr_memstage),
        .instruction_out(instruction_memstage)
    );

    //MEMORY STAGE
    memory_stage u_memorystage(
        .rst(rst),
        .load(load_memstage),
        .store(store_memstage),
        .op_b(op_b_memstage),
```

```
    .instruction(instruction_memstage),
    .alu_out_address(alu_res_out_memstage),
    .wrap_load_in(load_data_in),
    .mask(mask),
    .data_valid(data_mem_valid),
    .valid(instruc_mem_valid),
    .we_re(data_mem_we_re),
    .request(data_mem_request),
    .store_data_out(store_data_out),
    .wrap_load_out(wrap_load_memstage)
);

assign rd_memstage = instruction_memstage[11:7];
assign rd_wb = instruction_wb[11:7];
assign alu_out_address = alu_res_out_memstage;
assign mask_singal = mask ;
assign load_signal = load_memstage;

//MEMORY STAGE PIPELINE
memory_pipe u_memstagepipeline(
    .clk(clk),
    .mem_reg_in(mem_to_reg_memstage),
    .wrap_load_in(wrap_load_memstage),
    .alu_res(alu_res_out_memstage),
    .next_sel_addr(next_sel_address_memstage),
    .pre_address_in(pre_pc_addr_memstage),
    .instruction_in(instruction_memstage),
    .reg_write_in(reg_write_memstage),
    .reg_write_out(reg_write_wb),
    .alu_res_out(alu_res_out_wb),
    .mem_reg_out(mem_to_reg_wb),
    .next_sel_address(next_sel_address_wb),
    .instruction_out(instruction_wb),
    .pre_address_out(pre_pc_addr_wb),
    .wrap_load_out(wrap_load_wb)
);

//WRITE BACK STAGE
write_back u_wbstage(
    .mem_to_reg(mem_to_reg_wb),
    .alu_out(alu_res_out_wb),
    .data_mem_out(wrap_load_wb),
    .next_sel_address(next_sel_address_wb),
    .rd_sel_mux_out(rd_wb_data)
);
```

```
endmodule
```

Microprocessor.v

The microprocessor module is the top-level module that connects the core, instruction memory, and data memory together. It takes the clock and reset signals and distributes them to all submodules. It connects the program counter output from the core to the address input of the instruction memory to fetch instructions. The instruction memory sends the fetched instruction to the core for decoding and execution. The core generates control signals such as read/write, request, mask, and load signals for both memories. It also sends the ALU-generated address and store data to the data memory during store operations. During load operations, data memory sends the read data back to the core. Handshake signals (request and valid) are used to synchronize memory access. The external instruction input is used to write instructions into instruction memory. Overall, this module manages communication and data flow between the core instruction memory and memory blocks.

```
// microprocessor
module microprocessor (
    input wire clk,
    input wire rst,
    input wire [31:0]instruction
    );

    wire [31:0] instruction_data;
    wire [31:0] pc_address;
    wire [31:0] load_data_out;
    wire [31:0] alu_out_address;
    wire [31:0] store_data;
    wire [3:0]  mask;
    wire [3:0]  instruc_mask_singal;
    wire instruction_mem_we_re;
    wire instruction_mem_request;
    wire instruc_mem_valid;
    wire data_mem_valid;
    wire data_mem_we_re;
    wire data_mem_request;
    wire load_signal;
    wire store;

    // INSTRUCTION MEMORY
    instruc_mem_top #(
        .INIT_MEM(1)
    )u_instruction_memory(
        .clk(clk),
```

```
        .rst(rst),
        .we_re(instruction_mem_we_re),
        .request(instruction_mem_request),
        .mask(instruc_mask_singal),
        .address(pc_address[9:2]),
        .data_in(instruction),
        .valid(instruc_mem_valid),
        .data_out(instruction_data)
    );

    //CORE
    core u_core(
        .clk(clk),
        .rst(rst),
        .instruction(instruction_data),
        .load_data_in(load_data_out),
        .mask_singal(mask),
        .load_signal(load_signal),
        .instruc_mask_singal(instruc_mask_singal),
        .instruction_mem_we_re(instruction_mem_we_re),
        .instruction_mem_request(instruction_mem_request),
        .data_mem_we_re(data_mem_we_re),
        .data_mem_request(data_mem_request),
        .instruc_mem_valid(instruc_mem_valid),
        .data_mem_valid(data_mem_valid),
        .store_data_out(store_data),
        .pc_address(pc_address),
        .alu_out_address(alu_out_address)
    );

    // DATA MEMORY
    data_mem_top u_data_memory(
        .clk(clk),
        .rst(rst),
        .we_re(data_mem_we_re),
        .request(data_mem_request),
        .address(alu_out_address[9:2]),
        .data_in(store_data),
        .mask(mask),
        .load(load_signal),
        .valid(data_mem_valid),
        .data_out(load_data_out)
    );
endmodule
```